

Modular Scene Filtering via the Pixar Hydra 2.0 Architecture

Tom Cauchois
tcauchois@pixar.com
Pixar Animation Studios
USA

Steve LaVieta
stevell@pixar.com
Pixar Animation Studios
USA

ABSTRACT

Pixar’s Hydra project began as an abstract scene interface to an OpenGL-based renderer intended for interactive viewports across multiple applications. As the project added integrations with other renderers (including path tracers), it became clear that it needed a richer scene interface to convey a broader set of renderer features, as well as a more structured and modular way to resolve the scene into rendering primitives.

We’ll discuss the Hydra 2.0 architecture that was developed to address this problem, with examples from two scene filtering cases:

- Material network transformations in service of a specific renderer.
- Merging scene data from multiple sources in service of native application integration.

ACM Reference Format:

Tom Cauchois and Steve LaVieta. 2022. Modular Scene Filtering via the Pixar Hydra 2.0 Architecture. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH ’22 Talks)*, August 07–11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532836.3536280>

1 INTRODUCTION

Pixar’s Hydra project is a rendering framework that abstracts scene and renderer access through plugins. It can accept scene data from multiple heterogeneous scene graphs, and it can deliver the data to a user-selected renderer. Hydra is distributed with the OpenUSD [Elkoura et al. 2019] project, and ships with scene access to USD and renderer implementations through GL and RenderMan, but exists in a rich ecosystem of scene, renderer, and application integrations that build off of each other.

Hydra’s initial design was driven by the constraints of reading USD scenes into an OpenGL-based renderer. Because USD scenes are semantically rich, Hydra’s first scene abstraction, the **Scene Delegate**, condensed the scene down to a renderer-friendly representation by flattening the scene hierarchy and discarding **prims** (scene objects) and prim attributes that weren’t renderable. The Scene Delegate also became a natural place to implement necessary scene evaluation, such as tessellating implicit surfaces, regrouping instanced geometry, and altering materials.

As Hydra was extended to integrate with new renderers and workflows, the Scene Delegate’s fixed notions of how to flatten and

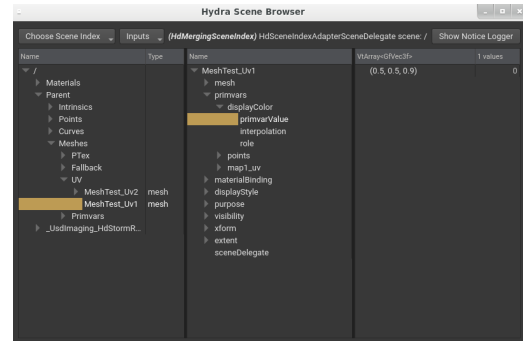


Figure 1: A UI display of Hydra’s scene view, inspectable after each evaluation step. ©Pixar

filter the scene became limiting; and as the complexity of scene evaluation performed by Hydra grew, we saw a need to decompose the evaluation steps into pluggable, inspectable, and self-contained operators. These two needs inspired the Hydra 2.0 API.

2 HYDRA 2.0 ARCHITECTURE OVERVIEW

Hydra now offers a scene abstraction called a **Scene Index**. The Scene Index API provides access to the full scene hierarchy and prim data on demand, letting renderers choose how to filter their view of the scene. It replaces the Scene Delegate’s attribute representation of C++ structs and bitfield invalidation flags with the **Data Source** API, which represents either a value or a container of other Data Sources. Nesting of Data Sources lets Hydra represent structured data like geometry topologies in an abstract, inspectable way, with a natural path structure to address attribute invalidations. Hydra expectations of structure are represented by schemas overlaid on Data Source containers, but these schemas don’t preclude other data from being present.

Scene evaluation and transformation can be expressed as a **Scene Index Filter**, which is a Scene Index that presents a procedurally modified view of another Scene Index. For example, a transform flattening Scene Index Filter applied to an input scene can pass input scene data through unmodified, except for replacing local transforms with composed transforms. The ability to decompose scene evaluation into separate modules makes it easier to add features by isolating transformations from each other. If applications and renderers need the scene prepared differently, they can customize which Scene Index Filters are applied (Figure 2). Since each Scene Index Filter presents a complete and consistent view of the scene, we can inspect the scene data before and after a filter to debug individual transformations (Figure 1).

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH ’22 Talks, August 07–11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9371-3/22/08.

<https://doi.org/10.1145/3532836.3536280>

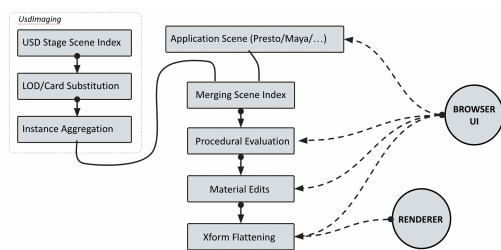


Figure 2: An example of how to decompose scene evaluation.
©Pixar

3 MATERIAL NETWORK TRANSFORMATIONS VIA SCENE INDEX FILTERS

Material network transformations are a common need in rendering pipelines. Material definitions within assets are often abstracted from the final form expected by a specific renderer. This may be to express a higher-level concept (like layering or projections) in a form more convenient for editing or for the use of a renderer-agnostic shading representation such as MaterialX.

These transformations don't belong in a Scene Delegate implementation, since they may vary by renderer or application; and implementing them within the renderer can be burdensome as the number of transformations grows. Using Scene Index Filters offers a configurable way to apply and reuse material transformations in an inspectable way without needing to modify the renderer itself.

3.1 MaterialX to RenderMan

The MaterialX specification [Foundation 2022a] includes Open Shading Language [Foundation 2022b] implementations for its standard library of pattern shading nodes. As RenderMan supports OSL for patterns, a Scene Index can replace or expand MaterialX shading nodes into equivalent OSL shaders. Those patterns serve as inputs to bxdf shading nodes which are specific to RenderMan. By separating this transformation of MaterialX-to-OSL into a Scene Index Filter, this transformation may be reused for any renderer with OSL pattern support.

3.2 High-level asset representation to optimized renderer representation

Scene Index Filters can be used to implement pipeline specific transforms such as replacing placeholder bxdf or pattern nodes or applying project-specific tweaks to a material, like those described in "UsdShade in the Pixar Pipeline" section 3.1 [Hecht et al. 2021].

4 SCENE DATA MERGING VIA SCENE INDEX FILTERS

Large scene workflows often make use of cached scene data references which serve as proxies for data which will be read into the renderer's scene directly rather than loaded natively by the host application. This is done for scalability as a host application's native object representation is usually much heavier than cached

equivalents – particularly when all data relevant to the render is present.

A trade-off is that the proxied scene data cannot be fully addressed by the editing tools of the host application. When those tools (such as animation rig evaluation) are needed, a common approach is to indicate that a subset of the proxied scene should be excluded in favor of equivalent natively loaded scene objects. This can present challenges both in scalability and correctness. If editing needs require large portions of the scene to be natively present, the advantages of proxied workflows are eroded. If the cache contains more information specific to the render (such as materials and rendering-specific attributes) than would be naturally present in the application native representation, the render may not be equivalent.

Scene Index Filters allow for the composition of native and proxied scene data at a fine granularity.

4.1 Merging Scene Index Filter

A Scene Index Filter which combines multiple input Scene Indices with potentially overlapping prims allows for a sparser application-native representation of a primitive to be underlaid with the full information present in the cached scene, by composing application-native attributes with cached attributes for prims that exist in both scenes. This composition can be configured to happen before or after other evaluation steps like transform or inheritance flattening.

For example, if a character rig needs to be edited and evaluated live to replace point positions for a subset of prims, only the prims and attributes necessary for that evaluation need to be loaded as native application data objects. The application's Scene Index can provide a sparse data representation of these prims, relying on the cached data Scene Index to provide other attributes like materials and rendering configuration. Note that the Scene Index providing cached data doesn't need to be aware of this usage; the merging Scene Index Filter can be configured to give the native representation a stronger opinion for whichever attributes are present.

5 ROADMAP

All Hydra releases since 22.03 use the Scene Index representation internally, with Hydra converting between Scene Delegate and Scene Index representations for API compatibility. These releases allow registration of per-renderer Scene Index Filters, which have enabled new features; and they allow for inspecting the Hydra scene view, which has been a powerful debugging tool. Work is in progress on Scene Index-native versions of our USD, OpenGL, and RenderMan integrations.

REFERENCES

- George Elkoura, Sebastian Grassia, Sunya Boonyatera, Alex Mohr, Pol Jeremias-Vila, and Matt Kuruc. 2019. A Deep Dive into Universal Scene Description and Hydra. In *ACM SIGGRAPH 2019 Courses* (Los Angeles, California) (SIGGRAPH '19). Association for Computing Machinery, New York, NY, USA, Article 1, 48 pages. <https://doi.org/10.1145/3305366.3328033>
- Academy Software Foundation. 2022a. MaterialX. <https://www.materialx.org/>.
- Academy Software Foundation. 2022b. Open Shading Language. <http://openshadinglanguage.org/>.
- Florian Hecht, Daniel McCoy, Stephen LaVietes, and F. Sebastian Grassia. 2021. UsdShade in the Pixar Pipeline. In *ACM SIGGRAPH 2021 Talks* (Virtual Event, USA) (SIGGRAPH '21). Association for Computing Machinery, New York, NY, USA, Article 5, 2 pages. <https://doi.org/10.1145/3450623.3464670>